Lesson 17：环境变量、配置管理与 API Key 安全

LLM Learn 系统课程

2026 年 11 月

目录

1 本课目标与工程背景 2

2 环境变量、.env 与优先级 2

3 类型验证与计算边界 2

4 API Key 安全与配置加载 2

5 环境安装与执行 3

6 源码分层与逐行解释 3

7 常见错误与处理 3

8 练习与客观验收 3

9 官方参考 4

10 完整源代码及中文逐行注释 4

11 测试与结果真实性 6

1 本课目标与工程背景

先修：Lesson 15 Responses API、Lesson 16 多轮聊天。建议学习时间 90–120 分钟。目标是独立管理模型名称、请求超时、Token 上限和 API Key，用离线测试验证安全边界。

错误的做法是在 Python 源码中直接写入 API Key 并反复修改模型配置。Git 历史可能保留秘密，多个开发环境也难以复用同一份代码。正确做法是将业务逻辑和配置分离。

2 环境变量、.env 与优先级

环境变量由操作系统提供给进程，Python 使用 os.getenv 读取。即使写入 30，读取后通常仍是字符串，需要类型转换。

使用 python-dotenv 的 load_dotenv(override=False)。系统已有的环境变量优先于.env，.env 优先于非敏感默认值。空字符串若不合法，应主动拒绝，不应静默当作缺失。

$$C(k) = \begin{cases} E(k), & k \in E, \\ D(k), & k \notin E,\ k \in D, \\ C_0(k), & \text{otherwise.} \end{cases}$$

```
# Windows PowerShell
$env:OPENAI_MODEL = "gpt-4.1-mini"
$env:LLM_TIMEOUT_SECONDS = "45"
# macOS/Linux: export LLM_TIMEOUT_SECONDS=45
```

3 类型验证与计算边界

AppConfig 是 dataclass(frozen=True),字段依次为模型 ID、timeout_seconds(秒)、max_output_tokens（整数）和 api_key。frozen=True 阻止意外重新赋值，但不是加密。

parse_timeout 先转换 float，再用 math.isfinite 排除 inf 和 nan；parse_max_tokens 转换为 int 并限制 1–8192。上下限是课程为防止参数错误制定的教学范围，并非模型规格。

$$0 < t_{\text{timeout}} \le 120 \text{ s}, \qquad 1 \le N_{\text{output}} \le 8192$$

4 API Key 安全与配置加载

本地.env.example 只存放示例字段，可上传 GitHub；真实.env 必须加入.gitignore，不得上传。开放的前端浏览器、手机客户端也不应该直接包含 API Key。

api_key 使用 field(repr=False)，防止普通 print(config) 把密钥打印出来；但属性本身仍可被访问，这不是加密，也不能替代权限控制、凭据轮换或秘密管理服务。

load_config 负责环境变量与.env 的合并及校验。require_api_key=False 允许本课纯离线练习；正常联网模式缺密钥会明确抛 ConfigError。错误文本只报告变量名，不输出密钥值。

5 环境安装与执行

默认.env 路径从源代码文件所在位置计算，而不依赖当前终端目录。这样 VS Code、命令行或 GitHub Actions 从不同工作目录执行，都能获得一致行为。

先安装 python-dotenv，然后使用 –offline 命令进行没有真实 Key 的配置检查。测试通过后再运行 8 项单元测试，并检查.env 是否被 Git 忽略。

```
# 在 llm-learn 仓库根目录运行 (Windows PowerShell)
py -m pip install -r lessons/lesson17/code/requirements.txt
Copy-Item lessons/lesson17/.env.example lessons/lesson17/.env
py lessons/lesson17/code/config.py --offline
py -m unittest discover -s lessons/lesson17/tests -v
git check-ignore lessons/lesson17/.env
```

6 源码分层与逐行解释

第一层:AppConfig 定义配置结构,has_api_key 返回是否已配置的布尔值;第二层:parse_timeout 与 parse_max_tokens 校验类型和允许范围；第三层：load_config 合并三个来源；第四层：main 只进行命令行解析和脱敏输出。

完整源码直接附在讲义末尾，与 code/config.py 逐字一致。模块、类、函数、关键语句都配有中文解释，方便对照函数参数、返回值、异常与工程作用。

7 常见错误与处理

ModuleNotFoundError: dotenv：安装本课 requirements.txt 并确认 IDE 的 Python 解释器正确。

缺少 OPENAI_API_KEY：本课用 –offline 可正常完成；真正在线调用前再配置有效密钥和 API 额度。

timeout 设置为 inf 或 nan：float 可以解析这些字符串，但 math.isfinite 会拒绝它们。Token 上限写成 1.5、0、8193 也应该被拒绝。

不要打印 os.environ 或 config.api_key,这些调试习惯会直接泄露凭据。如果密钥意外上传 GitHub，应尽快轮换或吊销并检查提交历史和账户用量。

8 练习与客观验收

练习 1：同时设置.env 的 30 秒和系统环境变量 45 秒，预测最终采用哪一个，并解释原因。

练习 2：分别设置 timeout 为 inf、nan、-1、0、121，解释为什么都不合法。

练习 3：设置 MAX_OUTPUT_TOKENS 为 0、500、8193、1.5，检查边界验证结果。

练习 4：使用 git check-ignore 确认.env 被排除，并确保.env.example 不含秘密。

练习 5：运行 8 项离线测试，解释默认值、dotenv 优先级、缺失 Key、repr 脱敏等测试到底验证了什么。

验收标准:能更改配置而不修改业务代码;不会泄漏密钥;离线 8 项测试通过;能够解释 repr=False 的安全作用与局限。

9 官方参考

python-dotenv:https://bbc2.github.io/python-dotenv/reference/;OpenAI API keys:https://developers.open keys; Python dataclasses: https://docs.python.org/3/library/dataclasses.html

10 完整源代码及中文逐行注释

```
"""Lesson 17: 从环境变量与 .env 安全加载 LLM 配置。

用途: 不访问网络、不调用付费 API; 可用 python code/config.py --offline 检查配置。
配置来源: 系统环境变量 > 本课本地 .env > 代码中的非敏感默认值。
安全限制: 不会打印完整 API Key; 测试只使用假的示例密钥。
"""

# argparse 将命令行选项转换为程序可读的参数。
import argparse
# math.isfinite 排除无穷大和 NaN 等非法超时值。
import math
# os.getenv 从进程环境中读取字符串配置。
import os
# dataclass 简化配置对象; field 控制私有字段的显示。
from dataclasses import dataclass, field
# Path 定位本课文件夹和 .env。
from pathlib import Path
# load_dotenv 读取本地配置文件, 默认不会覆盖系统变量。
from dotenv import load_dotenv


# 自定义异常区分配置错误与真实网络 API 错误。
class ConfigError(ValueError):
    """ 配置缺失、格式不正确或超出允许范围时抛出。"""


# frozen 防止创建后意外重新赋值, 但不提供密钥加密。
@dataclass(frozen=True)
class AppConfig:
    """ 经过验证的配置, 模型名、超时秒数、Token 上限和私有 Key。

    参数: model 为模型 ID; timeout_seconds 单位秒;
    max_output_tokens 为正整数; api_key 为不应打印的秘密。
    """

    # 选择有访问权限的模型 ID。
    model: str
    # 设置网络超时时间 (秒)。
    timeout_seconds: float
    # 控制大模型最大输出 Token 数。
    max_output_tokens: int
    # 防止普通 print(config) 将 Key 输出到日志。
    api_key: str = field(repr=False)
```

```
    # 属性让外部代码不用读取 Key 就知道是否已配置。
    @property
    def has_api_key(self) -> bool:
        """ 只返回是否存在凭据，不输出凭据本身。"""
        # 空字符串为 False，非空字符串为 True。
        return bool(self.api_key)


def parse_timeout(value: str) -> float:
    """ 把环境文本转换为 0–120 秒内的有限正数，否则抛 ConfigError。"""
    try:
        # 将字符串转换为 float。
        timeout = float(value)
    except ValueError as error:
        # 出错时只报告字段名称，不回显用户传入的原始内容。
        raise ConfigError("LLM_TIMEOUT_SECONDS 必须是数字") from error
    # 禁止 inf、nan、负值、零和高于 120 的超时。
    if not math.isfinite(timeout) or not 0 < timeout <= 120:
        # 返回安全的边界提示。
        raise ConfigError("LLM_TIMEOUT_SECONDS 必须在 0 到 120 秒之间")
    # 以秒为单位返回合法的超时配置。
    return timeout


def parse_max_tokens(value: str) -> int:
    """ 把环境文本转换为 1–8192 的整数 Token 上限，否则抛 ConfigError。"""
    try:
        # int 会拒绝 "1.5" 等非整数文本。
        tokens = int(value)
    except ValueError as error:
        # 报错只显示配置名称，避免泄露意外输入的秘密。
        raise ConfigError("LLM_MAX_OUTPUT_TOKENS 必须是整数") from error
    # 在调用收费 API 前检查教学用途允许的 Token 上限。
    if not 1 <= tokens <= 8192:
        # 过小或过大的值都明确拒绝。
        raise ConfigError("LLM_MAX_OUTPUT_TOKENS 必须介于 1 与 8192")
    # 返回已验证的最大输出 Token 数。
    return tokens


def load_config(env_file: Path | None = None, require_api_key: bool = True) -> AppConfig:
    """ 读取配置并执行验证。

    参数：env_file 默认为本课 .env；require_api_key 为是否强制要求 Key。
    返回：AppConfig。异常：ConfigError。
    """
    # 使用代码文件自身路径定位 lesson17，不依赖终端启动目录。
    lesson_folder = Path(__file__).resolve().parents[1]
    # 支持单元测试提供临时 .env 路径以隔离真实环境。
    dotenv_path = env_file if env_file is not None else lesson_folder / ".env"
    # override=False：系统已存在的环境变量保持最高优先级。
    load_dotenv(dotenv_path=dotenv_path, override=False)
    # 未配置模型时使用教学默认值，其他环境可通过 OPENAI_MODEL 覆盖。
    model = os.getenv("OPENAI_MODEL", "gpt-4.1-mini").strip()
    # 非空模型名称是发送请求之前的基本校验。
    if not model:
        # 不打印可疑输入值，只指出错误配置名。
        raise ConfigError("OPENAI_MODEL 不能为空")
```

```
    # 将文本超时转换为已经验证的正浮点数。
    timeout = parse_timeout(os.getenv("LLM_TIMEOUT_SECONDS", "30"))
    # 将 Token 上限转换为已经验证的正整数。
    max_tokens = parse_max_tokens(os.getenv("LLM_MAX_OUTPUT_TOKENS", "500"))
    # API Key 只从环境中读取，绝不能硬编码在源文件内。
    api_key = os.getenv("OPENAI_API_KEY", "").strip()
    # 正常联网应强制要求密钥，离线练习可明确关闭这一要求。
    if require_api_key and not api_key:
        # 安全报错不会暴露任何密钥内容。
        raise ConfigError(" 缺少 OPENAI_API_KEY；请使用环境变量或未提交的 .env")
    # 生成只读配置实例，后续业务代码按字段访问。
    return AppConfig(model, timeout, max_tokens, api_key)


def main() -> None:
    """CLI 配置检查入口；--offline 不要求 Key，也不连接网络。"""
    # 创建可以显示 --help 的命令行解析器。
    parser = argparse.ArgumentParser(description="Lesson 17: LLM 配置安全检查")
    # --offline 是不连接大模型服务的学习模式。
    parser.add_argument("--offline", action="store_true", help=" 允许不设置 API Key")
    # 获取解析后的布尔开关。
    options = parser.parse_args()
    try:
        # 离线模式允许无 Key，正常模式则要求存在 Key。
        config = load_config(require_api_key=not options.offline)
    except ConfigError as error:
        # 配置不合格时以非零退出码结束，而非报告运行成功。
        parser.exit(2, f" 配置错误：{error}\n")
    # 只打印可以公开的模型名称。
    print(f" 模型名称：{config.model}")
    # 打印以秒计量的超时时间。
    print(f" 超时：{config.timeout_seconds:g} 秒")
    # 打印最大输出 Token 上限。
    print(f" 最大输出：{config.max_output_tokens} tokens")
    # 只报告是否存在 Key，绝不打印 Key 本身。
    print(f"API Key 已配置：{'是' if config.has_api_key else '否'}")


if __name__ == "__main__":
    # 仅直接运行脚本时调用 CLI，import 时不自动执行。
    main()
```

11 测试与结果真实性

实际运行和 GitHub Actions 状态见 results/verification.md；本课不执行收费的模型调用。